

Tarea3. Implementación de la estructura completa de una base de datos (DDL)

Integrantes:

Jhon Byron Quiroz Perez

Curso: Bases de datos — **Grupo:** G100

Docente: Oralia Cortes Grajales

Institución universitaria Pascual Bravo

Medellin, 2026

Tarea3. Implementación de la estructura completa de una base de datos (DDL)..... 1
1. Identificación del SGBD..... 4
1.1 Ventajas y desventajas..... 4
1.2 Justificación de la elección..... 4
2. Descripción del modelo físico..... 6
2.1 Correspondencia de tipos de datos..... 6
2.2 Decisiones sobre los tipos de datos..... 6
2.3 Nomenclatura..... 7
3. Script DDL y su estructura..... 8
3.1 Secciones del script..... 8
3.2 Orden de creación..... 8
3.3 Estructura de cada tabla..... 8
4. Inventario de tablas..... 9
4.1 Diagrama del modelo físico..... 10
5. Descripción de las tablas..... 11
5.1 Catálogos (tablas sin claves foráneas)..... 11
5.2 Módulo perfil y conexiones..... 12
5.3 Módulo grupos..... 14
5.4 Módulo publicaciones..... 16
6. Implementación de las relaciones..... 21
7. Conclusiones individuales..... 24
8. Referencias..... 25

1. Identificación del SGBD

El SGBD elegido es **PostgreSQL 16**, administrado con **pgAdmin 4**. Para tomar la decisión se compararon cinco sistemas de gestión de bases de datos relacionales.

Tabla 1. Comparación de los SGBD considerados.

Criterio	PostgreSQL	MySQL	SQL Server	Oracle	SQLite
Licencia y costo	Libre (licencia PostgreSQL), sin costo	Edición Community libre (GPL); Oracle ofrece ediciones comerciales	Comercial; edición Express gratuita con límites de tamaño	Comercial; edición gratuita con límites	Dominio público
Apego al estándar SQL	Muy alto	Medio; varias extensiones propias	Alto, con dialecto T-SQL	Alto, con dialecto PL/SQL	Medio; tipado flexible
Restricciones CHECK	Sí, desde siempre	Se aplican solo desde la versión 8.0.16	Sí	Sí	Sí
Claves IDENTITY estándar	Sí (GENERATED AS IDENTITY)	No; usa AUTO_INCREMENT	Sí (IDENTITY propio)	Sí	No; usa ROWID
Herramienta gráfica	pgAdmin 4 (gratuita)	MySQL Workbench	SQL Server Management Studio	SQL Developer	DB Browser for SQLite
Sistemas operativos	Windows, Linux, macOS	Windows, Linux, macOS	Windows y Linux	Windows y Linux	Todos (archivo local)
Concurrencia y crecimiento	Alta (MVCC), apta para producción	Alta	Alta	Muy alta	Baja: pensada para un solo usuario

1.1 Ventajas y desventajas

PostgreSQL. *Ventajas:* es libre y sin costo de licencia; aplica rigurosamente las restricciones de integridad (CHECK, claves foráneas con todas las acciones referenciales, IDENTITY estándar); tiene tipos de datos precisos y permite esquemas para organizar los objetos; pgAdmin 4 genera el diagrama ER desde la base de datos. *Desventajas:* su configuración inicial (usuarios, puertos, autenticación) es menos inmediata que la de SQLite, y en algunos servicios de hosting compartido económicos es menos común que MySQL.

MySQL. *Ventajas:* muy popular, fácil de instalar y con mucha documentación. *Desventajas:* hasta la versión 8.0.16 aceptaba las restricciones CHECK sin aplicarlas, por lo que en versiones anteriores se pierden reglas del modelo; usa AUTO_INCREMENT en lugar del estándar IDENTITY.

SQL Server. *Ventajas:* herramientas muy completas y buena integración con el ecosistema de Microsoft. *Desventajas:* es comercial; la edición gratuita tiene límites de tamaño y su herramienta principal funciona solo en Windows.

1.2 Justificación de la elección

Se eligió **PostgreSQL 16** con **pgAdmin 4** como herramienta gráfica. La razón principal es que el modelo lógico depende de muchas reglas de integridad (22 restricciones CHECK, 34 claves foráneas con acciones distintas, claves compuestas y claves generadas), y PostgreSQL las aplica todas de forma estricta y con sintaxis estándar. Además, es gratuito, funciona en los sistemas operativos del equipo y puede llevarse a producción sin cambiar de producto si la red social crece.

2. Descripción del modelo físico

El modelo físico conserva la estructura del modelo lógico de la Tarea 2: las mismas 25 tablas, 102 columnas y 34 relaciones. La transformación consistió en tomar las decisiones que dependen del SGBD:

1. Cada tipo genérico (Texto, Entero, Lógico, Fecha, FechaHora) se convirtió en un tipo concreto de PostgreSQL con su tamaño.
2. Las claves sustitutas se implementaron como `INTEGER GENERATED ALWAYS AS IDENTITY`.
3. Cada clave y regla del diccionario de datos se escribió como una restricción con nombre (PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK).
4. Se definieron las acciones ON DELETE y ON UPDATE de cada clave foránea.

2.1 Correspondencia de tipos de datos

Tabla 2. Conversión de los tipos genéricos del modelo lógico a tipos de PostgreSQL.

Tipo lógico	Tipo en PostgreSQL	Dónde se usa	Justificación
Texto(n)	VARCHAR(n)	Nombres, descripciones, URL, códigos y listas de valores (60 columnas)	Conserva la longitud máxima definida en el diccionario; PostgreSQL rechaza valores más largos.
Texto(6) de formato fijo	CHAR(6)	`grupo_estudio.periodo_academico`	El periodo siempre tiene 6 caracteres (2026-2).
Entero (clave sustituta)	INTEGER GENERATED ALWAYS AS IDENTITY	5 claves primarias	Número consecutivo generado por la base de datos con sintaxis SQL estándar.
Entero (cantidad pequeña)	SMALLINT	`semestre`, `creditos`, `max_integrantes`, `cupo_maximo`	Valores acotados (máximo 32.767); ocupa 2 bytes en lugar de 4.
Entero (referencia o consecutivo)	INTEGER	Claves foráneas hacia claves IDENTITY y `num_comentario`	Debe coincidir con el tipo de la clave referenciada.
Lógico	BOOLEAN	`es_privado`, `esta_resuelta`	Solo admite verdadero o falso.
Fecha	DATE	4 columnas	Fechas sin hora: nacimiento, creación, unión, inicio.
FechaHora	TIMESTAMP	8 columnas	Momentos exactos: publicaciones, comentarios, inscripciones, horario de eventos.

2.2 Decisiones sobre los tipos de datos

VARCHAR(n) y no TEXT. En PostgreSQL ambos tienen el mismo rendimiento, pero VARCHAR(n) rechaza valores más largos que el máximo definido en el diccionario de datos, lo que agrega una validación. Ningún campo del modelo carece de un límite natural, por eso no se usó TEXT.

INTEGER y no BIGINT. INTEGER llega a 2.147.483.647, una cifra que una red social institucional no alcanzará. BIGINT duplicaría el espacio de cada clave y de cada clave foránea que la referencia sin ninguna ventaja.

SMALLINT para cantidades acotadas. El semestre va de 1 a 10 y los créditos, cupos e integrantes son números pequeños.

IDENTITY y no SERIAL. `GENERATED ALWAYS AS IDENTITY` es la forma del estándar SQL; SERIAL es propio de PostgreSQL. Con ALWAYS, además, nadie puede insertar un identificador manualmente.

TIMESTAMP y no TIMESTAMPTZ. La red social funciona en una sola zona horaria (Colombia), así que no es necesario guardar el desfase horario.

Códigos como texto. `codigo_estudiante`, `codigo_programa` y `codigo_materia` son VARCHAR porque son identificadores: pueden tener letras o ceros a la izquierda y nunca se operan aritméticamente.

2.3 Nomenclatura

Tabla 3. Reglas de nombres aplicadas en el modelo físico.

Objeto	Regla	Ejemplo
Esquema	Un esquema propio para el proyecto	`red_social`
Tablas	Minúsculas, singular, palabras separadas por guion bajo, sin tildes ni ñ	`estudiante`, `grupo_estudio`
Columnas	Minúsculas con guion bajo; claves sustitutas con `id_`, claves naturales con `codigo_`	`id_grupo`, `codigo_materia`
Claves foráneas	Nombradas por el rol que cumplen cuando el nombre de la clave padre no basta	`codigo_autor`, `codigo_seguidor`
Restricciones	`pk_`, `fk_`, `uq_` o `ck_` + tabla + columna o rol	`pk_estudiante`, `fk_grupo_creador`
Índices	`ix_` + tabla + columna	`ix_membresia_grupo`

3. Script DDL y su estructura

El modelo físico está implementado en el archivo **`01_ddl_red_social_pascualina.sql`**, ubicado en la carpeta `tarea3/script` del repositorio. Al ejecutarlo en pgAdmin 4 crea el esquema completo.

3.1 Secciones del script

Tabla 4. Organización interna del script DDL.

Sección	Qué hace
Encabezado	Identifica el proyecto, el SGBD, el autor, la estructura del archivo y la nomenclatura de restricciones.
1. Esquema	`DROP SCHEMA IF EXISTS red_social CASCADE` y `CREATE SCHEMA red_social`, para que el script se pueda ejecutar varias veces y siempre deje la base de datos en el mismo estado. `SET search_path` hace que las tablas se creen dentro del esquema.
2. Tablas	Las 25 sentencias `CREATE TABLE`, agrupadas en cinco subsecciones por módulo.
3. Índices	17 sentencias `CREATE INDEX` sobre claves foráneas, porque PostgreSQL no los crea automáticamente para ellas.
4. Comentarios	25 sentencias `COMMENT ON TABLE` que guardan la descripción de cada tabla en la base de datos.

3.2 Orden de creación

Una clave foránea solo puede apuntar a una tabla que ya existe. Por eso las tablas se crean de la más independiente a la más dependiente:

Tabla 5. Orden de creación de las tablas.

Subsección del script	Tablas	Dependen de
2.1 Catálogos (tablas sin claves foráneas)	`programa_academico`, `materia`, `interes`, `habilidad`	Ninguna tabla
2.2 Módulo perfil y conexiones	`estudiante`, `enlace_perfil`, `estudiante_interes`, `estudiante_habilidad`, `seguimiento`, `mentoria`	`programa_academico`, `interes`, `habilidad`
2.3 Módulo grupos	`grupo`, `grupo_estudio`, `equipo_hackaton`, `club_interes`, `membresia`	`estudiante`, `materia`, `interes`
2.4 Módulo publicaciones	`publicacion`, `publicacion_etiqueta`, `pregunta`, `recurso`, `noticia`, `meme`, `comentario`, `reaccion`	`estudiante`, `grupo`, `materia`
2.5 Módulo eventos	`evento`, `inscripcion`	`estudiante`, `grupo`

3.3 Estructura de cada tabla

Todas las tablas siguen la misma estructura: primero las columnas con su tipo, su obligatoriedad (NULL o NOT NULL) y su valor por defecto; después, las restricciones con nombre. Por ejemplo:

```
-- estudiante: Estudiantes registrados en la red social con los datos de su perfil.
CREATE TABLE estudiante (
    codigo_estudiante     VARCHAR(15)   NOT NULL,
    nombres               VARCHAR(60)   NOT NULL,
    apellidos             VARCHAR(60)   NOT NULL,
    correo_institucional  VARCHAR(100)  NOT NULL,
    fecha_nacimiento      DATE          NOT NULL,
    semestre              SMALLINT      NOT NULL,
    biografia             VARCHAR(500)  NULL,
    foto_perfil           VARCHAR(255)  NULL,
    fecha_registro        TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    codigo_programa       VARCHAR(10)   NOT NULL,
    CONSTRAINT pk_estudiante PRIMARY KEY (codigo_estudiante),
    CONSTRAINT uq_estudiante_correo UNIQUE (correo_institucional),
    CONSTRAINT ck_estudiante_semestre CHECK (semestre BETWEEN 1 AND 10),
    CONSTRAINT ck_estudiante_correo CHECK (correo_institucional LIKE '%_@_%._%'),
    CONSTRAINT fk_estudiante_programa FOREIGN KEY (codigo_programa) REFERENCES programa_academico
(codigo_programa)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
```

4. Inventario de tablas

Tabla 6. Inventario de las 25 tablas del esquema `red_social`, en orden de creación.

#	Tabla	Módulo	Clave primaria	Columnas	Claves foráneas
1	programa_academico	Perfil	codigo_programa	4	0
2	materia	Compartido	codigo_materia	3	0
3	interes	Perfil	id_interes	3	0
4	habilidad	Perfil	id_habilidad	3	0
5	estudiante	Perfil	codigo_estudiante	10	1
6	enlace_perfil	Perfil	codigo_estudiante, url_enlace	2	1
7	estudiante_interes	Perfil	codigo_estudiante, id_interes	2	2
8	estudiante_habilidad	Perfil	codigo_estudiante, id_habilidad	3	2
9	seguimiento	Conexiones	codigo_seguidor, codigo_seguido	3	2
10	mentoria	Conexiones	codigo_mentor, codigo_aprendiz	5	2
11	grupo	Grupos	id_grupo	7	1
12	grupo_estudio	Grupos	id_grupo	3	2
13	equipo_hackaton	Grupos	id_grupo	4	1
14	club_interes	Grupos	id_grupo	2	2
15	membresia	Grupos	codigo_estudiante, id_grupo	4	2
16	publicacion	Publicaciones	id_publicacion	7	2
17	publicacion_etiqueta	Publicaciones	id_publicacion, etiqueta	2	1
18	pregunta	Publicaciones	id_publicacion	3	2
19	recurso	Publicaciones	id_publicacion	3	1
20	noticia	Publicaciones	id_publicacion	3	1
21	meme	Publicaciones	id_publicacion	2	1
22	comentario	Publicaciones	id_publicacion, num_comentario	5	2
23	reaccion	Publicaciones	codigo_estudiante, id_publicacion	4	2
24	evento	Eventos	id_evento	11	2
25	inscripcion	Eventos	codigo_estudiante, id_evento	4	2

4.1 Diagrama del modelo físico

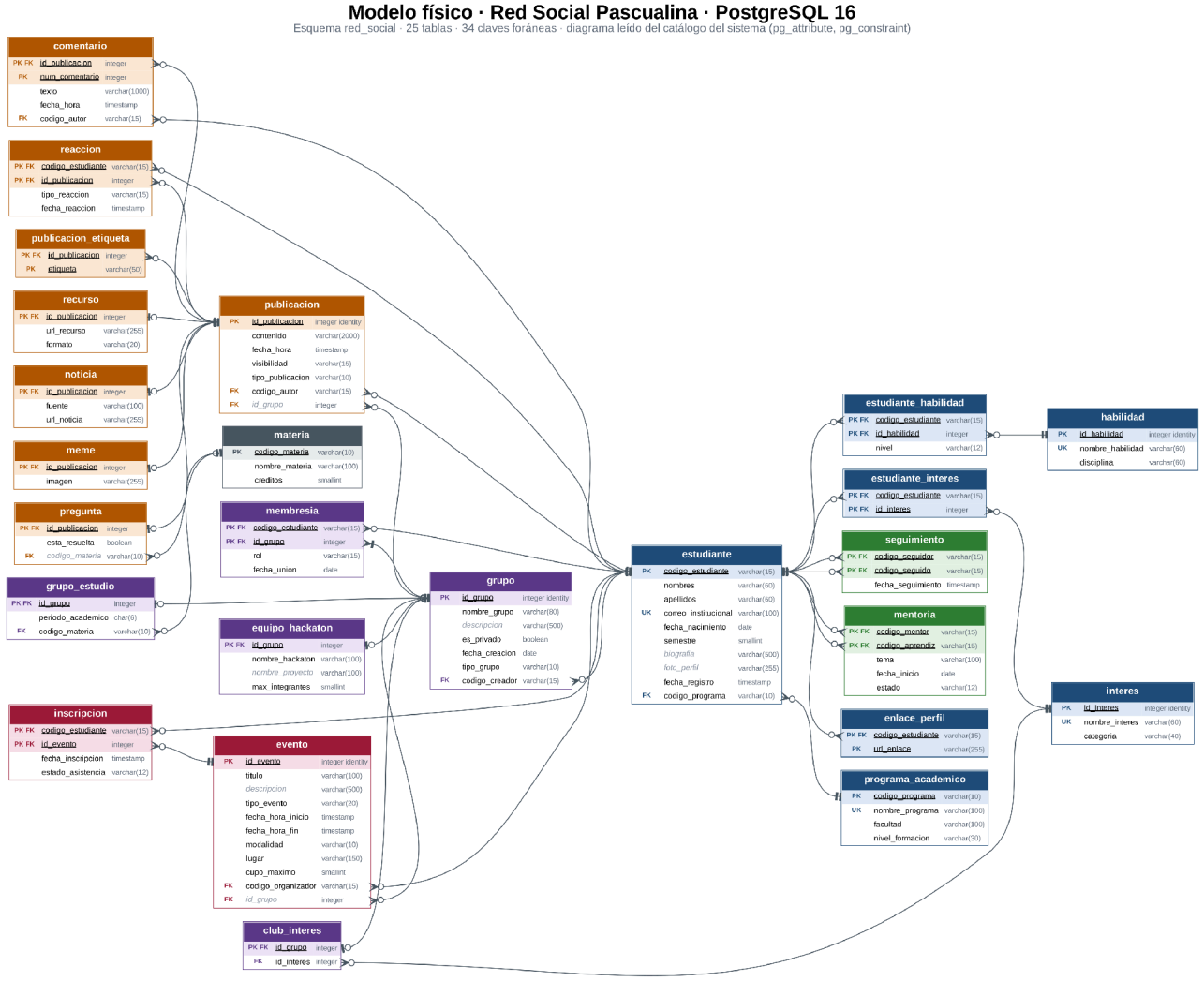


Figura 1. *Diagrama del modelo físico del esquema `red_social` en PostgreSQL. PK, FK y UK identifican las claves; las columnas en gris admiten nulos. Las líneas usan notación pata de gallo: ‖ exactamente uno, ○| cero o uno, ○< cero o muchos.*

5. Descripción de las tablas

Para cada tabla se listan sus campos con el tipo de dato y el tamaño, se identifica la clave primaria y se justifica su elección y el tipo de dato de los campos clave.

5.1 Catálogos (tablas sin claves foráneas)

Tabla `programa_academico`

Programas académicos de la institución (área de estudio del estudiante).

Campo	Tipo y tamaño	Clave	Descripción
`codigo_programa`	VARCHAR(10)	PK	Código institucional del programa.
`nombre_programa`	VARCHAR(100)	UNIQUE	Nombre oficial del programa.
`facultad`	VARCHAR(100)	—	Facultad a la que pertenece el programa.
`nivel_formacion`	VARCHAR(30)	—	Técnico profesional, Tecnológico o Profesional universitario.

Clave primaria: `codigo_programa`. Clave **natural**: el código del programa ya es único y estable en la institución (por ejemplo, 'TDS'), así que no hace falta inventar otro identificador y la clave tiene significado para quien lee los datos.

Tipo de los campos clave: VARCHAR(10) y no INTEGER, porque es un identificador y no una cantidad: puede contener letras (TDS) o ceros a la izquierda y nunca se suma ni se promedia. VARCHAR con longitud y no TEXT, porque el código tiene un tamaño máximo conocido y la longitud funciona como una validación adicional.

Tabla `materia`

Asignaturas del plan de estudios.

Campo	Tipo y tamaño	Clave	Descripción
`codigo_materia`	VARCHAR(10)	PK	Código oficial de la asignatura.
`nombre_materia`	VARCHAR(100)	—	Nombre de la asignatura.
`creditos`	SMALLINT	—	Créditos académicos.

Clave primaria: `codigo_materia`. Clave **natural**: el código de la asignatura ya es único y estable en la institución (por ejemplo, 'BD1'), así que no hace falta inventar otro identificador y la clave tiene significado para quien lee los datos.

Tipo de los campos clave: VARCHAR(10) y no INTEGER, porque es un identificador y no una cantidad: puede contener letras (BD1) o ceros a la izquierda y nunca se suma ni se promedia. VARCHAR con longitud y no TEXT, porque el código tiene un tamaño máximo conocido y la longitud funciona como una validación adicional.

Tabla `interes`

Catálogo de intereses (inteligencia artificial, videojuegos, fútbol…).

Campo	Tipo y tamaño	Clave	Descripción
`id_interes`	INTEGER	PK	Identificador del interés.
`nombre_interes`	VARCHAR(60)	UNIQUE	Nombre del interés.
`categoria`	VARCHAR(40)	—	Categoría del interés: tecnología, deporte, arte, ocio…

Clave primaria: `id_interes`. Clave **sustituta** generada por PostgreSQL con `GENERATED ALWAYS AS IDENTITY`: un interés no tiene un código natural (su nombre se puede repetir o cambiar), así que la base de datos le asigna un número consecutivo.

Tipo de los campos clave: INTEGER y no BIGINT: INTEGER admite más de 2.100 millones de valores, muchísimo más de los registros que tendrá una red social institucional, y ocupa 4 bytes en lugar de 8, tanto en esta tabla como en cada clave foránea que la referencia. Se eligió IDENTITY y no SERIAL porque es SQL estándar y, con `ALWAYS`, impide que alguien inserte un id manualmente por error.

Tabla `habilidad`

Catálogo de habilidades en disciplinas o deportes.

Campo	Tipo y tamaño	Clave	Descripción
`id_habilidad`	INTEGER	PK	Identificador de la habilidad.
`nombre_habilidad`	VARCHAR(60)	UNIQUE	Nombre de la habilidad (Python, diseño gráfico, voleibol…).
`disciplina`	VARCHAR(60)	—	Disciplina o deporte al que pertenece la habilidad.

Clave primaria: `id_habilidad`. Clave **sustituta** generada por PostgreSQL con `GENERATED ALWAYS AS IDENTITY`: una habilidad no tiene un código natural (su nombre se puede repetir o cambiar), así que la base de datos le asigna un número consecutivo.

Tipo de los campos clave: INTEGER y no BIGINT: INTEGER admite más de 2.100 millones de valores, muchísimo más de los registros que tendrá una red social institucional, y ocupa 4 bytes en lugar de 8, tanto en esta tabla como en cada clave foránea que la referencia. Se eligió IDENTITY y no SERIAL porque es SQL estándar y, con `ALWAYS`, impide que alguien inserte un id manualmente por error.

5.2 Módulo perfil y conexiones

Tabla `estudiante`

Estudiantes registrados en la red social con los datos de su perfil.

Campo	Tipo y tamaño	Clave	Descripción
`codigo_estudiante`	VARCHAR(15)	PK	Código estudiantil asignado por la institución.
`nombres`	VARCHAR(60)	—	Nombres del estudiante.
`apellidos`	VARCHAR(60)	—	Apellidos del estudiante.
`correo_institucional`	VARCHAR(100)	UNIQUE	Correo electrónico institucional.
`fecha_nacimiento`	DATE	—	Fecha de nacimiento.
`semestre`	SMALLINT	—	Semestre que cursa (1 a 10).
`biografia`	VARCHAR(500)	—	Texto breve de presentación del perfil.
`foto_perfil`	VARCHAR(255)	—	Ruta o URL de la foto de perfil.
`fecha_registro`	TIMESTAMP	—	Momento en que se creó el perfil.
`codigo_programa`	VARCHAR(10)	FK → programa_academico	Programa académico que cursa.

Clave primaria: `codigo_estudiante`. Clave **natural**: el código estudiantil ya es único y estable en la institución (por ejemplo, '20241001'), así que no hace falta inventar otro identificador y la clave tiene significado para quien lee los datos.

Tipo de los campos clave: VARCHAR(15) y no INTEGER, porque es un identificador y no una cantidad: puede contener letras (algunos códigos incluyen prefijos) o ceros a la izquierda y nunca se suma ni se

promedia. VARCHAR con longitud y no TEXT, porque el código tiene un tamaño máximo conocido y la longitud funciona como una validación adicional. El correo también es único, pero se dejó como UNIQUE y no como clave primaria porque un correo sí puede cambiar.

Tabla `enlace_perfil`

Enlaces externos del perfil (GitHub, LinkedIn, portafolio).

Campo	Tipo y tamaño	Clave	Descripción
`codigo_estudiante`	VARCHAR(15)	PK FK → estudiante	Estudiante dueño del enlace.
`url_enlace`	VARCHAR(255)	PK	Dirección del enlace.

Clave primaria: `codigo_estudiante`, `url_enlace`. Clave **compuesta** (`codigo_estudiante`, `url_enlace`): la tabla nace de un atributo multivaluado, así que cada fila es «un enlace de un estudiante». La clave impide registrar dos veces el mismo enlace para la misma persona.

Tipo de los campos clave: VARCHAR(15) igual que `estudiante.codigo_estudiante`, y VARCHAR(255) para la URL, la longitud habitual de una dirección web.

Tabla `estudiante_interes`

Intereses registrados por cada estudiante.

Campo	Tipo y tamaño	Clave	Descripción
`codigo_estudiante`	VARCHAR(15)	PK FK → estudiante	Estudiante.
`id_interes`	INTEGER	PK FK → interes	Interés registrado.

Clave primaria: `codigo_estudiante`, `id_interes`. Clave **compuesta** (`codigo_estudiante`, `id_interes`): es la tabla intermedia de una relación M:N y cada fila representa una pareja única. La propia clave impide registrar dos veces la misma pareja, sin necesitar una columna `id` adicional.

Tipo de los campos clave: Cada columna de la clave tiene exactamente el mismo tipo que la clave primaria a la que apunta (VARCHAR(15) para los estudiantes, INTEGER para las tablas con IDENTITY). Las columnas INTEGER de esta tabla no son IDENTITY, porque aquí no se generan: se copian de la tabla padre.

Tabla `estudiante_habilidad`

Habilidades que domina cada estudiante y su nivel.

Campo	Tipo y tamaño	Clave	Descripción
`codigo_estudiante`	VARCHAR(15)	PK FK → estudiante	Estudiante.
`id_habilidad`	INTEGER	PK FK → habilidad	Habilidad.
`nivel`	VARCHAR(12)	—	Básico, Intermedio o Avanzado.

Clave primaria: `codigo_estudiante`, `id_habilidad`. Clave **compuesta** (`codigo_estudiante`, `id_habilidad`): es la tabla intermedia de una relación M:N y cada fila representa una pareja única. La propia clave impide registrar dos veces la misma pareja, sin necesitar una columna `id` adicional.

Tipo de los campos clave: Cada columna de la clave tiene exactamente el mismo tipo que la clave primaria a la que apunta (VARCHAR(15) para los estudiantes, INTEGER para las tablas con IDENTITY). Las columnas INTEGER de esta tabla no son IDENTITY, porque aquí no se generan: se copian de la tabla padre.

Tabla `seguimiento`

Quién sigue a quién dentro de la red.

Campo	Tipo y tamaño	Clave	Descripción
`codigo_seguidor`	VARCHAR(15)	PK FK → estudiante	Estudiante que sigue (rol seguidor).
`codigo_seguido`	VARCHAR(15)	PK FK → estudiante	Estudiante seguido (rol seguido).
`fecha_seguimiento`	TIMESTAMP	—	Momento en que empezó a seguirlo.

Clave primaria: `codigo_seguidor`, `codigo_seguido`. Clave **compuesta** (`codigo_seguidor`, `codigo_seguido`): es la tabla intermedia de una relación M:N y cada fila representa una pareja única. La propia clave impide registrar dos veces la misma pareja, es decir, seguir dos veces a la misma persona, sin necesitar una columna `id` adicional.

Tipo de los campos clave: Cada columna de la clave tiene exactamente el mismo tipo que la clave primaria a la que apunta (VARCHAR(15) para los estudiantes, INTEGER para las tablas con IDENTITY). Las columnas INTEGER de esta tabla no son IDENTITY, porque aquí no se generan: se copian de la tabla padre.

Tabla `mentoria`

Mentorías entre estudiantes de semestres avanzados y sus aprendices.

Campo	Tipo y tamaño	Clave	Descripción
`codigo_mentor`	VARCHAR(15)	PK FK → estudiante	Estudiante mentor.
`codigo_aprendiz`	VARCHAR(15)	PK FK → estudiante	Estudiante aprendiz.
`tema`	VARCHAR(100)	—	Tema o materia de la mentoría.
`fecha_inicio`	DATE	—	Fecha en que comenzó la mentoría.
`estado`	VARCHAR(12)	—	Activa o Finalizada.

Clave primaria: `codigo_mentor`, `codigo_aprendiz`. Clave **compuesta** (`codigo_mentor`, `codigo_aprendiz`): es la tabla intermedia de una relación M:N y cada fila representa una pareja única. La propia clave impide registrar dos veces la misma pareja, sin necesitar una columna `id` adicional.

Tipo de los campos clave: Cada columna de la clave tiene exactamente el mismo tipo que la clave primaria a la que apunta (VARCHAR(15) para los estudiantes, INTEGER para las tablas con IDENTITY). Las columnas INTEGER de esta tabla no son IDENTITY, porque aquí no se generan: se copian de la tabla padre.

5.3 Módulo grupos

Tabla `grupo`

Grupos creados en la red (datos comunes a los tres tipos).

Campo	Tipo y tamaño	Clave	Descripción
`id_grupo`	INTEGER	PK	Identificador del grupo.
`nombre_grupo`	VARCHAR(80)	—	Nombre visible del grupo.
`descripcion`	VARCHAR(500)	—	Propósito del grupo.
`es_privado`	BOOLEAN	—	Verdadero si unirse requiere aprobación.
`fecha_creacion`	DATE	—	Fecha de creación del grupo.

Campo	Tipo y tamaño	Clave	Descripción
tipo_grupo	VARCHAR(10)	—	Estudio, Hackaton o Club.
codigo_creador	VARCHAR(15)	FK → estudiante	Estudiante que creó el grupo.

Clave primaria: id_grupo. Clave **sustituta** generada por PostgreSQL con GENERATED ALWAYS AS IDENTITY: un grupo no tiene un código natural (su nombre se puede repetir o cambiar), así que la base de datos le asigna un número consecutivo.

Tipo de los campos clave: INTEGER y no BIGINT: INTEGER admite más de 2.100 millones de valores, muchísimo más de los registros que tendrá una red social institucional, y ocupa 4 bytes en lugar de 8, tanto en esta tabla como en cada clave foránea que la referencia. Se eligió IDENTITY y no SERIAL porque es SQL estándar y, con ALWAYS, impide que alguien inserte un id manualmente por error.

Tabla grupo_estudio

Grupos de estudio por materia.

Campo	Tipo y tamaño	Clave	Descripción
id_grupo	INTEGER	PK FK → grupo	Misma clave del grupo general.
periodo_academico	CHAR(6)	—	Periodo en formato AAAA-P (por ejemplo, 2026-2).
codigo_materia	VARCHAR(10)	FK → materia	Materia en la que se enfoca el grupo.

Clave primaria: id_grupo. Clave **compartida con la superclase**: id_grupo es a la vez clave primaria y clave foránea hacia grupo. Como es clave primaria no puede repetirse, y eso garantiza la relación 1:1 entre cada fila de la subclase y su fila en grupo.

Tipo de los campos clave: INTEGER, el mismo tipo de grupo.id_grupo. No es IDENTITY porque el valor no se genera aquí: se toma de la superclase.

Tabla equipo_hackaton

Equipos para hackatones.

Campo	Tipo y tamaño	Clave	Descripción
id_grupo	INTEGER	PK FK → grupo	Misma clave del grupo general.
nombre_hackaton	VARCHAR(100)	—	Hackatón en la que participa.
nombre_proyecto	VARCHAR(100)	—	Proyecto que desarrolla el equipo.
max_integrantes	SMALLINT	—	Número máximo de integrantes.

Clave primaria: id_grupo. Clave **compartida con la superclase**: id_grupo es a la vez clave primaria y clave foránea hacia grupo. Como es clave primaria no puede repetirse, y eso garantiza la relación 1:1 entre cada fila de la subclase y su fila en grupo.

Tipo de los campos clave: INTEGER, el mismo tipo de grupo.id_grupo. No es IDENTITY porque el valor no se genera aquí: se toma de la superclase.

Tabla club_interes

Clubes de interés (videojuegos, programación competitiva…).

Campo	Tipo y tamaño	Clave	Descripción
id_grupo	INTEGER	PK FK → grupo	Misma clave del grupo general.

Campo	Tipo y tamaño	Clave	Descripción
id_interes	INTEGER	FK → interes	Interés sobre el que trata el club.

Clave primaria: id_grupo. Clave **compartida con la superclase**: id_grupo es a la vez clave primaria y clave foránea hacia grupo. Como es clave primaria no puede repetirse, y eso garantiza la relación 1:1 entre cada fila de la subclase y su fila en grupo.

Tipo de los campos clave: INTEGER, el mismo tipo de grupo.id_grupo. No es IDENTITY porque el valor no se genera aquí: se toma de la superclase.

Tabla membresia

Miembros de cada grupo y su rol.

Campo	Tipo y tamaño	Clave	Descripción
codigo_estudiante	VARCHAR(15)	PK FK → estudiante	Estudiante miembro.
id_grupo	INTEGER	PK FK → grupo	Grupo.
rol	VARCHAR(15)	—	Administrador, Moderador o Miembro.
fecha_union	DATE	—	Fecha en que se unió al grupo.

Clave primaria: codigo_estudiante, id_grupo. Clave **compuesta** (codigo_estudiante, id_grupo): es la tabla intermedia de una relación M:N y cada fila representa una pareja única. La propia clave impide registrar dos veces la misma pareja, es decir, unir dos veces al mismo estudiante al mismo grupo, sin necesitar una columna id adicional.

Tipo de los campos clave: Cada columna de la clave tiene exactamente el mismo tipo que la clave primaria a la que apunta (VARCHAR(15) para los estudiantes, INTEGER para las tablas con IDENTITY). Las columnas INTEGER de esta tabla no son IDENTITY, porque aquí no se generan: se copian de la tabla padre.

5.4 Módulo publicaciones

Tabla publicacion

Publicaciones de los estudiantes (datos comunes a todos los tipos).

Campo	Tipo y tamaño	Clave	Descripción
id_publicacion	INTEGER	PK	Identificador de la publicación.
contenido	VARCHAR(2000)	—	Texto de la publicación.
fecha_hora	TIMESTAMP	—	Momento de la publicación.
visibilidad	VARCHAR(15)	—	Pública, Seguidores o Grupo.
tipo_publicacion	VARCHAR(10)	—	General, Pregunta, Recurso, Noticia o Meme.
codigo_autor	VARCHAR(15)	FK → estudiante	Estudiante autor.
id_grupo	INTEGER	FK → grupo	Grupo en el que se publicó.

Clave primaria: id_publicacion. Clave **sustituta** generada por PostgreSQL con GENERATED ALWAYS AS IDENTITY: una publicación no tiene un código natural (su nombre se puede repetir o cambiar), así que la base de datos le asigna un número consecutivo.

Tipo de los campos clave: INTEGER y no BIGINT: INTEGER admite más de 2.100 millones de valores, muchísimo más de los registros que tendrá una red social institucional, y ocupa 4 bytes en lugar de 8, tanto en esta tabla como en cada clave foránea que la referencia. Se eligió IDENTITY y no SERIAL porque es SQL estándar y, con ALWAYS, impide que alguien inserte un id manualmente por error.

Tabla `publicacion_etiqueta`

Etiquetas (#BD1, #Python…) de cada publicación.

Campo	Tipo y tamaño	Clave	Descripción
`id_publicacion`	INTEGER	PK FK → publicacion	Publicación etiquetada.
`etiqueta`	VARCHAR(50)	PK	Texto de la etiqueta, sin el símbolo #.

Clave primaria: `id_publicacion`, `etiqueta`. Clave **compuesta** (`id_publicacion`, `etiqueta`): proviene del atributo multivaluado `etiquetas`. La clave impide repetir la misma etiqueta en una publicación.

Tipo de los campos clave: INTEGER igual que `publicacion.id_publicacion`, y VARCHAR(50) para la etiqueta, suficiente para palabras clave cortas.

Tabla `pregunta`

Preguntas sobre tareas.

Campo	Tipo y tamaño	Clave	Descripción
`id_publicacion`	INTEGER	PK FK → publicacion	Misma clave de la publicación general.
`esta_resuelta`	BOOLEAN	—	Verdadero si la pregunta ya fue respondida.
`codigo_materia`	VARCHAR(10)	FK → materia	Materia sobre la que trata la pregunta.

Clave primaria: `id_publicacion`. Clave **compartida con la superclase**: `id_publicacion` es a la vez clave primaria y clave foránea hacia `publicacion`. Como es clave primaria no puede repetirse, y eso garantiza la relación 1:1 entre cada fila de la subclase y su fila en `publicacion`.

Tipo de los campos clave: INTEGER, el mismo tipo de `publicacion.id_publicacion`. No es IDENTITY porque el valor no se genera aquí: se toma de la superclase.

Tabla `recurso`

Recursos útiles compartidos.

Campo	Tipo y tamaño	Clave	Descripción
`id_publicacion`	INTEGER	PK FK → publicacion	Misma clave de la publicación general.
`url_recurso`	VARCHAR(255)	—	Enlace al material.
`formato`	VARCHAR(20)	—	PDF, Video, Repositorio, Página web u Otro.

Clave primaria: `id_publicacion`. Clave **compartida con la superclase**: `id_publicacion` es a la vez clave primaria y clave foránea hacia `publicacion`. Como es clave primaria no puede repetirse, y eso garantiza la relación 1:1 entre cada fila de la subclase y su fila en `publicacion`.

Tipo de los campos clave: INTEGER, el mismo tipo de `publicacion.id_publicacion`. No es IDENTITY porque el valor no se genera aquí: se toma de la superclase.

Tabla `noticia`

Noticias de la industria tecnológica.

Campo	Tipo y tamaño	Clave	Descripción
`id_publicacion`	INTEGER	PK FK → publicacion	Misma clave de la publicación general.

Campo	Tipo y tamaño	Clave	Descripción
fuente	VARCHAR(100)	—	Medio que publicó la noticia.
url_noticia	VARCHAR(255)	—	Enlace a la nota original.

Clave primaria: id_publicacion. Clave **compartida con la superclase**: id_publicacion es a la vez clave primaria y clave foránea hacia publicacion. Como es clave primaria no puede repetirse, y eso garantiza la relación 1:1 entre cada fila de la subclase y su fila en publicacion.

Tipo de los campos clave: INTEGER, el mismo tipo de publicacion.id_publicacion. No es IDENTITY porque el valor no se genera aquí: se toma de la superclase.

Tabla meme

Memes de la vida universitaria.

Campo	Tipo y tamaño	Clave	Descripción
id_publicacion	INTEGER	PK FK → publicacion	Misma clave de la publicación general.
imagen	VARCHAR(255)	—	Ruta de la imagen del meme.

Clave primaria: id_publicacion. Clave **compartida con la superclase**: id_publicacion es a la vez clave primaria y clave foránea hacia publicacion. Como es clave primaria no puede repetirse, y eso garantiza la relación 1:1 entre cada fila de la subclase y su fila en publicacion.

Tipo de los campos clave: INTEGER, el mismo tipo de publicacion.id_publicacion. No es IDENTITY porque el valor no se genera aquí: se toma de la superclase.

Tabla comentario

Comentarios escritos en las publicaciones.

Campo	Tipo y tamaño	Clave	Descripción
id_publicacion	INTEGER	PK FK → publicacion	Publicación comentada.
num_comentario	INTEGER	PK	Consecutivo del comentario dentro de la publicación.
texto	VARCHAR(1000)	—	Contenido del comentario.
fecha_hora	TIMESTAMP	—	Momento en que se escribió.
codigo_autor	VARCHAR(15)	FK → estudiante	Estudiante autor del comentario.

Clave primaria: id_publicacion, num_comentario. Clave **compuesta de entidad débil** (id_publicacion, num_comentario): el comentario no tiene identificador propio; se identifica por la publicación a la que pertenece más un consecutivo (clave parcial).

Tipo de los campos clave: INTEGER para ambas columnas: id_publicacion porque copia la clave de publicacion, y num_comentario porque una publicación muy comentada podría superar el máximo de SMALLINT (32.767).

Tabla reaccion

Reacciones de los estudiantes a las publicaciones.

Campo	Tipo y tamaño	Clave	Descripción
codigo_estudiante	VARCHAR(15)	PK FK → estudiante	Estudiante que reacciona.

Campo	Tipo y tamaño	Clave	Descripción
id_publicacion	INTEGER	PK FK → publicacion	Publicación a la que reacciona.
tipo_reaccion	VARCHAR(15)	—	Me gusta, Útil, Divertido o Me encanta.
fecha_reaccion	TIMESTAMP	—	Momento de la reacción.

Clave primaria: codigo_estudiante, id_publicacion. Clave **compuesta** (codigo_estudiante, id_publicacion): es la tabla intermedia de una relación M:N y cada fila representa una pareja única. La propia clave impide registrar dos veces la misma pareja, con lo que se cumple la regla de una sola reacción por estudiante y publicación, sin necesitar una columna id adicional.

Tipo de los campos clave: Cada columna de la clave tiene exactamente el mismo tipo que la clave primaria a la que apunta (VARCHAR(15) para los estudiantes, INTEGER para las tablas con IDENTITY). Las columnas INTEGER de esta tabla no son IDENTITY, porque aquí no se generan: se copian de la tabla padre.

5.5 Módulo eventos

Tabla evento

Reuniones de estudio, talleres y actividades sociales.

Campo	Tipo y tamaño	Clave	Descripción
id_evento	INTEGER	PK	Identificador del evento.
titulo	VARCHAR(100)	—	Nombre del evento.
descripcion	VARCHAR(500)	—	Detalle del evento.
tipo_evento	VARCHAR(20)	—	Reunión de estudio, Taller o Actividad social.
fecha_hora_inicio	TIMESTAMP	—	Inicio del evento.
fecha_hora_fin	TIMESTAMP	—	Fin del evento.
modalidad	VARCHAR(10)	—	Presencial, Virtual o Híbrida.
lugar	VARCHAR(150)	—	Salón o enlace de la videollamada.
cupo_maximo	SMALLINT	—	Número máximo de inscritos.
codigo_organizador	VARCHAR(15)	FK → estudiante	Estudiante que organiza el evento.
id_grupo	INTEGER	FK → grupo	Grupo que convoca el evento.

Clave primaria: id_evento. Clave **sustituta** generada por PostgreSQL con GENERATED ALWAYS AS IDENTITY: un evento no tiene un código natural (su nombre se puede repetir o cambiar), así que la base de datos le asigna un número consecutivo.

Tipo de los campos clave: INTEGER y no BIGINT: INTEGER admite más de 2.100 millones de valores, muchísimo más de los registros que tendrá una red social institucional, y ocupa 4 bytes en lugar de 8, tanto en esta tabla como en cada clave foránea que la referencia. Se eligió IDENTITY y no SERIAL porque es SQL estándar y, con ALWAYS, impide que alguien inserte un id manualmente por error.

Tabla inscripcion

Inscripciones de estudiantes a eventos.

Campo	Tipo y tamaño	Clave	Descripción
codigo_estudiante	VARCHAR(15)	PK FK → estudiante	Estudiante inscrito.

Campo	Tipo y tamaño	Clave	Descripción
`id_evento`	INTEGER	PK FK → evento	Evento.
`fecha_inscripcion`	TIMESTAMP	—	Momento de la inscripción.
`estado_asistencia`	VARCHAR(12)	—	Inscrito, Asistió o No asistió.

Clave primaria: `codigo_estudiante`, `id_evento`. Clave **compuesta** (`codigo_estudiante`, `id_evento`): es la tabla intermedia de una relación M:N y cada fila representa una pareja única. La propia clave impide registrar dos veces la misma pareja, es decir, inscribir dos veces al mismo estudiante en el mismo evento, sin necesitar una columna `id` adicional.

Tipo de los campos clave: Cada columna de la clave tiene exactamente el mismo tipo que la clave primaria a la que apunta (VARCHAR(15) para los estudiantes, INTEGER para las tablas con IDENTITY). Las columnas INTEGER de esta tabla no son IDENTITY, porque aquí no se generan: se copian de la tabla padre.

6. Implementación de las relaciones

Cada relación se implementó con una restricción FOREIGN KEY en la tabla hija que apunta a la clave primaria de la tabla padre (34 en total). **ON DELETE** define qué pasa con las filas hijas al borrar la fila padre: CASCADE las borra, RESTRICT impide el borrado y SET NULL deja la clave foránea en nulo. **ON UPDATE** define qué pasa si cambia el valor de la clave padre.

Para ON UPDATE se aplicó una regla uniforme: **CASCADE** cuando la clave padre es natural (`estudiante`, `programa_academico`, `materia`), porque esos códigos podrían corregirse y el cambio debe propagarse; **RESTRICT** cuando la clave padre es IDENTITY (`interes`, `habilidad`, `publicacion`, `grupo`, `evento`), porque esos identificadores no deben cambiar.

Tabla 7. Relaciones del modelo físico con sus acciones referenciales y su justificación.

Relación	Tabla padre → hija	Clave foránea	ON DELETE	ON UPDATE	Justificación
CURSA (N:1)	programa_academico → estudiante	`codigo_programa`	RESTRICT	CASCADE	RESTRICT al borrar: si se pudiera eliminar un programa, sus estudiantes quedarían sin programa o se borrarían con él. CASCADE al actualizar: si la institución cambia el código del programa, el cambio llega solo a sus estudiantes.
Multivaluado enlaces_perfil	estudiante → enlace_perfil	`codigo_estudiante`	CASCADE	CASCADE	CASCADE al borrar: un enlace no tiene sentido sin su dueño. CASCADE al actualizar porque el código estudiantil es una clave natural que podría corregirse.
TIENE_INTERES (M:N)	estudiante → estudiante_interes	`codigo_estudiante`	CASCADE	CASCADE	CASCADE: si se elimina el estudiante, se eliminan sus intereses registrados; la asociación deja de tener sentido.
TIENE_INTERES (M:N)	interes → estudiante_interes	`id_interes`	CASCADE	RESTRICT	CASCADE al borrar: si se retira un interés del catálogo, desaparece de los perfiles. RESTRICT al actualizar porque la clave IDENTITY no debe cambiar.
DOMINA (M:N)	estudiante → estudiante_habilidad	`codigo_estudiante`	CASCADE	CASCADE	CASCADE: las habilidades registradas pertenecen al perfil del estudiante.
DOMINA (M:N)	habilidad → estudiante_habilidad	`id_habilidad`	CASCADE	RESTRICT	CASCADE al borrar una habilidad del catálogo; RESTRICT al actualizar (clave IDENTITY).
SIGUE (M:N recursiva)	estudiante → seguimiento	`codigo_seguidor`	CASCADE	CASCADE	CASCADE: si se elimina un estudiante, desaparecen las relaciones en las que seguía a otros.
SIGUE (M:N recursiva)	estudiante → seguimiento	`codigo_seguido`	CASCADE	CASCADE	CASCADE: también desaparecen las relaciones en las que otros lo seguían.
MENTOREA (M:N recursiva)	estudiante → mentoria	`codigo_mentor`	CASCADE	CASCADE	CASCADE: una mentoría no puede existir sin su mentor.
MENTOREA (M:N recursiva)	estudiante → mentoria	`codigo_aprendiz`	CASCADE	CASCADE	CASCADE: ni sin su aprendiz.

Relación	Tabla padre → hija	Clave foránea	ON DELETE	ON UPDATE	Justificación
CREA (1:N)	estudiante → grupo	`codigo_creador`	RESTRICT	CASCADE	RESTRICT al borrar: con CASCADE, eliminar la cuenta de una persona borraría grupos completos con otros miembros, publicaciones y eventos. Antes hay que transferir o cerrar el grupo. CASCADE al actualizar el código estudiantil.
Especialización de GRUPO (1:1)	grupo → grupo_estudio	`id_grupo`	CASCADE	RESTRICT	CASCADE: al borrar un grupo se borra su fila de subclase.
ENFOCADO_EN (N:1)	materia → grupo_estudio	`codigo_materia`	RESTRICT	CASCADE	RESTRICT al borrar: no se puede eliminar una materia que tiene grupos de estudio, porque la participación del grupo es total (la FK es NOT NULL). CASCADE al actualizar el código de la materia.
Especialización de GRUPO (1:1)	grupo → equipo_hackaton	`id_grupo`	CASCADE	RESTRICT	CASCADE: la fila de subclase desaparece con su grupo.
Especialización de GRUPO (1:1)	grupo → club_interes	`id_grupo`	CASCADE	RESTRICT	CASCADE: la fila de subclase desaparece con su grupo.
TRATA_SOBRE (N:1)	interes → club_interes	`id_interes`	RESTRICT	RESTRICT	RESTRICT al borrar: no se puede retirar del catálogo un interés que define un club (participación total). RESTRICT al actualizar (clave IDENTITY).
PERTENECE (M:N)	estudiante → membresia	`codigo_estudiante`	CASCADE	CASCADE	CASCADE: si se elimina el estudiante, se eliminan sus membresías.
PERTENECE (M:N)	grupo → membresia	`id_grupo`	CASCADE	RESTRICT	CASCADE: si se elimina el grupo, se eliminan sus membresías. RESTRICT al actualizar (clave IDENTITY).
PUBLICA (1:N)	estudiante → publicacion	`codigo_autor`	CASCADE	CASCADE	CASCADE: las publicaciones son contenido del autor; si su cuenta se elimina, se elimina su contenido. CASCADE al actualizar el código estudiantil.
SE_PUBLICA_EN (N:1)	grupo → publicacion	`id_grupo`	CASCADE	RESTRICT	Admite nulos (publicación en el muro general). CASCADE al borrar: las publicaciones internas de un grupo desaparecen con él. RESTRICT al actualizar (clave IDENTITY).
Multivaluado etiquetas	publicacion → publicacion_etiqueta	`id_publicacion`	CASCADE	RESTRICT	CASCADE: las etiquetas desaparecen con su publicación.
Especialización de PUBLICACION (1:1)	publicacion → pregunta	`id_publicacion`	CASCADE	RESTRICT	CASCADE: la fila de subclase desaparece con su publicación.
REFERENTE_A (N:1)	materia → pregunta	`codigo_materia`	SET NULL	CASCADE	SET NULL al borrar: si se elimina la materia, la pregunta se conserva como pregunta general, porque la columna admite nulos. CASCADE al actualizar el código de la materia.
Especialización de PUBLICACION (1:1)	publicacion → recurso	`id_publicacion`	CASCADE	RESTRICT	CASCADE: la fila de subclase desaparece con su publicación.

Relación	Tabla padre → hija	Clave foránea	ON DELETE	ON UPDATE	Justificación
Especialización de PUBLICACION (1:1)	publicacion → noticia	`id_publicacion`	CASCADE	RESTRICT	CASCADE: la fila de subclase desaparece con su publicación.
Especialización de PUBLICACION (1:1)	publicacion → meme	`id_publicacion`	CASCADE	RESTRICT	CASCADE: la fila de subclase desaparece con su publicación.
RECIBE (identificadora, 1:N)	publicacion → comentario	`id_publicacion`	CASCADE	RESTRICT	CASCADE: el comentario es una entidad débil y no existe sin su publicación.
ESCRIBE (1:N)	estudiante → comentario	`codigo_autor`	CASCADE	CASCADE	CASCADE: si se elimina el autor, se eliminan sus comentarios.
REACCIONA (M:N)	estudiante → reaccion	`codigo_estudiante`	CASCADE	CASCADE	CASCADE: las reacciones desaparecen con el estudiante.
REACCIONA (M:N)	publicacion → reaccion	`id_publicacion`	CASCADE	RESTRICT	CASCADE: y también con la publicación.
ORGANIZA (1:N)	estudiante → evento	`codigo_organizador`	RESTRICT	CASCADE	RESTRICT al borrar: un evento con inscritos no debe desaparecer porque se elimine la cuenta de su organizador; primero se reasigna o cancela. CASCADE al actualizar el código estudiantil.
CONVOCA (1:N)	grupo → evento	`id_grupo`	SET NULL	RESTRICT	SET NULL al borrar: si se elimina el grupo, el evento se conserva como evento independiente (la columna admite nulos). RESTRICT al actualizar (clave IDENTITY).
SE_INSCRIBE (M:N)	estudiante → inscripcion	`codigo_estudiante`	CASCADE	CASCADE	CASCADE: las inscripciones desaparecen con el estudiante.
SE_INSCRIBE (M:N)	evento → inscripcion	`id_evento`	CASCADE	RESTRICT	CASCADE: y también con el evento.

7. Conclusiones individuales

Dificultades. La parte más difícil fue decidir las acciones referenciales. En el modelo lógico una relación era solo una línea entre dos tablas, pero en el DDL tuve que pensar qué pasa con los datos cuando algo se borra. Al principio quería usar CASCADE en todo, hasta que me di cuenta de que borrar a un estudiante podía eliminar grupos completos con otros miembros. También me costó ordenar la creación de las tablas: `publicacion` fallaba si se creaba antes que `grupo`.

Dudas. Me quedan dudas sobre cómo implementar las reglas que el DDL no alcanza a cubrir, como el cupo máximo de los eventos o que solo los miembros publiquen en un grupo. Entiendo que se resuelven con disparadores, pero todavía no los he trabajado.

Aspectos relevantes. Comprobé que el modelo físico no es solo copiar el modelo lógico: cada tipo de dato es una decisión (VARCHAR o TEXT, INTEGER o BIGINT) con efectos en el espacio y en la validación. Al ejecutar el script en pgAdmin y ver que PostgreSQL rechazaba un semestre 15 o una reacción duplicada, entendí que las restricciones protegen la información aunque la aplicación tenga errores.

8. Referencias

Canepa, G. (2016). *PostgreSQL database handbook*. http://dev.cs.ovgu.de/db/PostgreSQL-Database-Handbook.pdf

Elmasri, R. y Navathe, S. B. (2007). Algoritmos de diseño de bases de datos relacionales y dependencias adicionales. En *Fundamentos de sistemas de bases de datos* (5.ª ed., pp. 317–344). Pearson Educación.

Hsu, L. y Obe, R. (2015). *Essential PostgreSQL*. DZone Refcardz. https://ugoproto.github.io/ugodoc/pdf/52492-rc071_postgresql_2.pdf

Mannino, M. V. (2007). *Administración de bases de datos: diseño y desarrollo de aplicaciones* (3.ª ed.). McGraw-Hill Interamericana.

PostgreSQL Global Development Group. (s. f.). *PostgreSQL 16 documentation*. https://www.postgresql.org/docs/16/